BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent stores a lookup table mapping every possible percept sequence to an optimal action. In complex environments like Chess, the number of possible states and percept sequences grows exponentially—a phenomenon known as combinatorial explosion.

As the agent's lifetime increases,

The length of the percept sequence history grows longer with every step.

The lookup table required to store actions for every sequence grows exponentially large, making the table
physical impossible to store in memory or construct within a realistic timeframe.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Rule 1
(if percept['food_here']: -> return 'Stay'):

Condition: Sensors detect food at the current location (percept['food_here'] == True).
Action: The agent executes 'Stay'.

Rule 2
(if percept['wall_ahead']: -> return 'TurnLeft'):

Condition: Sensors detect an obstacle directly ahead (percept['wall_ahead'] == True).
Action: The agent executes 'TurnLeft'.

Rule 3
(return 'Forward'):

Condition: Default / catch-all condition when no explicit environmental conditions are met (food_here is False and wall_ahead is False).
Action: The agent executes 'Forward'.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Partial Observability: The agent only perceives local booleans (like wall_ahead: True or food_here: False) rather than its global coordinates (x, y). Distinct locations on the grid—such as two different corners—produce identical local sensory inputs.

Lack of Percept History: Because the agent has no memory (__init__) or percept sequence history, it cannot recall past visits, actions taken, or previous loop states.

Infinite Loop: Because the rules are purely deterministic, receiving the same local percept wall_ahead: True always triggers 'TurnLeft'. Moving 'Forward' in a bounded or U-shaped space leads to another wall where wall_ahead: True occurs again. Lacking memory to detect that it is revisiting a previously explored square, the agent cycles through 'TurnLeft' and 'Forward' indefinitely.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Transition Model (How the world evolves):
Implemented in update_state(). This method models how taking actions changes the agent's internal state over time.

Turning actions ('TurnLeft', 'TurnRight') update orientation state (self.facing).

Movement actions ('Forward') update spatial state (self.position = (x, y + 1) etc.) relative to current
orientation, provided percept['wall_ahead'] is False.

Navigating across the grid continuously updates the historical search trajectory state (self.path.append(self.position)).

Sensor Model (How sensory inputs reflect state):
Implemented in get_unvisited_neighbors(), backtrack(), and sense_and_act(). The agent integrates incoming sensor observations with its stored state memory.

Wall sensors (percept['wall_left'], percept['wall_up'], etc.) are mapped to surrounding coordinates relative to
self.position.

The agent filters out obstacles and checks coordinates against self.visited_cells. When trapped with no
unvisited options, backtrack() evaluates self.path[-2] against current coordinates to navigate backward,
effectively avoiding infinite cycles.

Finalize and Lock Lab 02 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING